



**FUSEMACHINES INC.
AI FELLOWSHIP PROGRAM
KATHMANDU, NEPAL**

**A FAIRNESS AND BIAS AUDIT SYSTEM PROPOSAL REPORT
ON
BIASAPERTURE: A DIAGNOSTIC AND EVALUATIVE FRAMEWORK FOR
AUDITING DEMOGRAPHIC BIAS IN FACIAL ANALYSIS SYSTEMS**

Submitted By:

Aaradhya Dev Tamrakar
Tisha Manandhar

Submitted To:

Fusemachines AI Fellowship 2026
Shreejan Kisee, Teaching Assistant, Fusemachines AI Fellowship
Kathmandu, Nepal

July 2026

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to Shreejan Kisee, Teaching Assistant, Fusemachines AI Fellowship at the Fusemachines AI Fellowship 2026, for supervising this proposal and for the detailed review of the preceding feasibility study that shaped the requirements and architecture presented in this report. We are grateful to Fusemachines for the structure and mentorship provided through the AI Fellowship programme, within which this capstone project is undertaken.

We also thank each other for the collaborative work that went into the requirement analysis, architectural design, and project planning documented here, and everyone who offered feedback during the drafting of this proposal.

Aaradhya Dev Tamrakar

Tisha Manandhar

ABSTRACT

Automated facial analysis systems are widely deployed despite well-documented accuracy disparities across demographic subgroups, and independent, reproducible auditing of these systems remains rare in practice. This report proposes BiasAperture, a diagnostic and evaluative software platform that computes subgroup and intersectional fairness metrics for a third-party facial-analysis model and reports them in a standardised, regulator-legible format. The platform is organised into five cooperating modules covering data ingestion, model interfacing, fairness-metric computation, explainability, and report generation. Its analytical core computes four disparity metrics, demographic parity difference, equalized odds difference, equal opportunity difference, and disparate impact ratio, using AIF360 and Fairlearn as independent, cross-validating backends, with every reported disparity accompanied by a chi-squared significance test and a bootstrap confidence interval rather than a bare threshold. A SHAP-based explainability layer attributes flagged disparities to input features, distinguishing genuinely demographic effects from unrelated confounds. Findings are assembled into an exportable, Model-Cards- and Datasheets-structured report in which every metric is traced to its specific basis under Article 10 of the EU AI Act and the corresponding function of the NIST AI Risk Management Framework. The design is validated against the FairFace and UTKFace benchmark datasets, with a FairFace-trained convolutional-network baseline serving as the platform’s own case study. BiasAperture is scoped strictly as diagnostic: it identifies and statistically characterises disparities but does not mitigate bias, retrain models, or generate synthetic demographic data.

Keywords: Algorithmic Fairness, Bias Auditing, Demographic Parity, Disparate Impact, EU AI Act, Explainable AI, Facial Analysis, Fairness Metrics, NIST AI RMF, SHAP

CONTENTS

ACKNOWLEDGEMENTS	i
ABSTRACT	ii
List of Figures	v
List of Tables	vi
LIST OF ABBREVIATIONS	vii
LIST OF SYMBOLS	viii
1. Introduction	1
1.1. Background	1
1.2. Problem Statement	2
1.3. Objectives	2
1.4. Scope and Limitations	3
2. Literature Review	4
3. Requirement Analysis	10
3.1. Functional Requirements	10
3.2. Non-Functional Requirements	11
3.3. System Requirements	12
4. System Architecture and Methodology	13
4.1. Design Principles	13
4.2. Architectural Modules	14
4.3. End-to-End Workflow of the System	17
4.4. Design Patterns	19
4.5. Regulatory Traceability Mapping	19
4.6. Work Breakdown Structure	20
4.7. Project Plan and Schedule	20
4.8. Descoping Strategy	23
4.9. Verification and Validation Strategy	23
5. Conclusion	25
5.1. Summary	25
5.2. Future Work	25
APPENDICES	27

A. Project Budget	27
A.1. Cloud Computing and Development Services	27
A.2. Documentation and Miscellaneous	27
B. Project Timeline	28
C. Locked Internal Schema	29
D. Risk Register	30
REFERENCES	32

LIST OF FIGURES

Figure 4-1. BiasAperture system architecture.	14
Figure 4-2. BiasAperture end-to-end audit workflow.	18
Figure 4-3. BiasAperture project schedule, work packages WP1–WP5.	22
Figure B-1. BiasAperture project schedule, work packages WP1–WP5 (reproduced from Chapter 4).	28

LIST OF TABLES

Table 2-1. Summary of the literature reviewed, highlighting key contributions and motivations for this work.	7
Table 4-1. Architectural modules and their responsibilities.	15
Table 4-2. Design patterns mapped to architectural modules.	19
Table 4-3. Article 10 components mapped to BiasAperture outputs.	20
Table 4-4. Work breakdown structure.	21
Table 4-5. Descoping order if the project falls behind schedule.	23
Table A-1. Cloud computing and development services cost.	27
Table A-2. Documentation and miscellaneous cost.	27
Table C-1. Dataset / test-matrix schema.	29
Table C-2. Metrics-dictionary schema, the shape every Fairness Metrics Computation Engine output must satisfy.	29
Table D-1. Project risk register: likelihood, impact, and mitigation strategy. . . .	31

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
CLI	Command-Line Interface
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSV	Comma-Separated Values
DIR	Disparate Impact Ratio
DPD	Demographic Parity Difference
EOD	Equalized Odds Difference
EOP	Equal Opportunity Difference
EU	European Union
F1	F1 Score
FR	Functional Requirement
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
JSON	JavaScript Object Notation
NFR	Non-Functional Requirement
NIST	National Institute of Standards and Technology
PDF	Portable Document Format
RAM	Random-Access Memory
RMF	Risk Management Framework
SHAP	SHapley Additive exPlanations
SOLID	Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, Dependency Inversion
UI	User Interface
VRAM	Video Random-Access Memory

LIST OF SYMBOLS

α	Significance threshold for chi-squared testing of independence between the predicted-label distribution and the demographic subgroup ($\alpha = 0.05$, NFR-001)
n	Subgroup sample size; the minimum threshold below which a metric is flagged as statistically insufficient rather than reported ($n \geq 30$, NFR-003)

1. INTRODUCTION

1.1. Background

Automated facial analysis systems, covering perceived gender classification, age estimation, and race or ethnicity prediction, are now embedded across commercial and public-sector applications ranging from photo tagging and demographic analytics to surveillance and access control. As these systems have proliferated, a substantial body of empirical research has documented that many of them exhibit statistically significant accuracy disparities across demographic subgroups, particularly by skin tone, gender presentation, and age. The foundational audit by Buolamwini and Gebru demonstrated that leading commercial gender-classification services performed markedly worse on darker-skinned female subjects than on lighter-skinned male subjects, with error rates as high as 34.7% for the worst-performing intersectional subgroup against as low as 0.8% for the best-performing one [1]. That finding catalysed a wider research agenda into fairness auditing for computer vision, subsequently surveyed comprehensively by Dehdashtian et al., who catalogue the sources of bias that arise across the machine learning pipeline, from data collection and annotation through model training and deployment, and the corresponding families of fairness metrics and mitigation techniques proposed in response [2].

Despite this growing awareness, independent, systematic, and reproducible auditing of facial analysis models remains comparatively rare in practice. Most organisations that deploy such systems either do not audit them for demographic bias at all, or rely on ad hoc, non-standardised evaluations that are difficult to reproduce, compare, or scrutinise externally. Dehdashtian et al.’s survey confirms that while a rich taxonomy of fairness metrics and mitigation techniques exists in the computer vision literature, comparatively little attention has been paid to standardising the auditing workflow itself into a repeatable pipeline [2].

This gap is also a regulatory one. The European Union (EU) Artificial Intelligence (AI) Act (EU Regulation 2024/1689) establishes binding data-governance, representativeness, and bias-detection obligations under Article 10 (Data and Data Governance) for providers of high-risk AI systems, a category that explicitly includes biometric categorisation [3]. The Digital Omnibus on AI (EU Regulation 2026/1744, in force 27 July 2026) subsequently deferred the compliance deadline for Annex III high-risk systems, including biometric categorisation, from 2 August 2026 to 2 December 2027 [4]. The obligations themselves are therefore not imminent but are settled and scheduled, giving providers a defined runway rather than removing the requirement: organisations that embed documentation and measurement infrastructure into their development lifecycle well ahead of the 2027 deadline avoid the substantially higher retrofit costs of doing so

under deadline pressure later. The National Institute of Standards and Technology (NIST) AI Risk Management Framework (Risk Management Framework (RMF)) offers a complementary, voluntary structure, organised around four iterative functions, Govern, Map, Measure, and Manage, for operationalising exactly this kind of risk assessment [5].

1.2. Problem Statement

There is therefore a clear engineering gap between a mature and growing body of fairness metrics, benchmark datasets, and bias-mitigation research, and the availability of an accessible, reusable, standards-aligned software platform that operationalises this research into a repeatable auditing workflow for facial analysis systems specifically. Existing audits, including the Gender Shades study itself, are typically conducted externally, manually, and for a single point in time; they are not designed as reusable, repeatable software artefacts that other researchers or organisations can apply to arbitrary models. This leaves model owners, auditors, and researchers without a systematic way to evaluate a facial-analysis model’s performance across demographic subgroups using standardised benchmark datasets, statistically grounded fairness metrics, and a transparent, shareable report, at precisely the moment regulatory frameworks such as the EU AI Act begin to require exactly this kind of evidence.

1.3. Objectives

1.3.1. General Objective

To design and propose an end-to-end diagnostic and evaluative software platform, BiasAperture, that systematically identifies demographic accuracy disparities in third-party facial analysis models and reports them in a standardised, regulator-legible format.

1.3.2. Specific Objectives

1. To design a modular software architecture that accepts a trained facial analysis model, or a file of its precomputed predictions, together with a demographically labelled benchmark dataset, and computes subgroup and intersectional fairness metrics from it.
2. To integrate established open-source fairness toolkits, AIF360 and Fairlearn, alongside standard statistical significance testing, so that every disparity the platform reports is methodologically defensible rather than an artefact of sampling noise.
3. To generate a standardised, exportable fairness report, structurally informed by the Model Cards [6] and Datasheets for Datasets [7] documentation conventions, that communicates subgroup performance disparities in a form accessible to both technical and non-technical stakeholders.

4. To validate the platform’s design against publicly available, demographically annotated benchmark datasets, principally FairFace [8] and UTKFace.
5. To map every computed fairness metric to a specific obligation under Article 10 and Annex IV of the EU AI Act and to the corresponding function of the NIST AI RMF, so the platform’s output is directly usable as compliance evidence.

1.4. Scope and Limitations

1.4.1. Scope

BiasAperture is scoped as a strictly diagnostic and evaluative platform. In scope are: (i) demographic subgroup and intersectional performance evaluation of a supplied facial-analysis classifier; (ii) computation of four disparity metrics, Demographic Parity Difference (DPD), Equalized Odds Difference (EOD), Equal Opportunity Difference (EOP), and Disparate Impact Ratio (DIR), backed by chi-squared significance testing and bootstrap confidence intervals; (iii) SHapley Additive exPlanations (SHAP)-based explainability to help identify which visual features are associated with a detected disparity; and (iv) automated generation of a Model-Cards- and Datasheets-style HyperText Markup Language (HTML) fairness report, with each metric traceable to its EU AI Act and NIST AI RMF basis. The platform is validated against the FairFace [8] and UTKFace benchmark datasets, with a FairFace-trained Convolutional Neural Network (CNN) baseline serving as the minimum case study.

1.4.2. Limitations

Consistent with these boundaries, BiasAperture does not attempt to mitigate bias, retrain or fine-tune models, or generate synthetic demographic data; its function is strictly diagnostic, not corrective. It reports where disparities exist and their statistical strength, but it does not prescribe or apply any remediation. Its findings are only as reliable as the label quality of the benchmark dataset supplied, a known limitation for UTKFace’s model-estimated age labels in particular, and every reported subgroup below a minimum sample size of $n \geq 30$ is flagged as statistically insufficient rather than scored. Full-dataset evaluation is bounded by a documented runtime target rather than guaranteed instantaneous; and if project timeline pressure requires descoping, the platform’s own cut-list (Chapter 4) defines, in order, which secondary capabilities are dropped first, while the diagnostic core, ingestion, one model interface, the fairness engine, one report format, and the scope-boundary statement itself, is never cut.

2. LITERATURE REVIEW

Chapter 1 established that a mature body of fairness research has not yet converged into a reusable auditing platform. To discuss the works that shape BiasAperture’s design, this chapter reviews each contributing paper in turn, what it studied, what we learned from it, and how our project adopts, extends, or departs from it, covering the formal definitions behind the Core Four metrics, the critiques that justify pairing every disparity with a significance test, the finding that motivates the explainability layer, the conventions that shape the report structure, the benchmark datasets against which the platform is validated, and the regulatory-technical bridge its output is designed to evidence. A summary table at the end of the chapter consolidates these contributions and the concepts BiasAperture adopts from each.

Buolamwini and Gebru’s Gender Shades study audited commercial gender-classification services and found that they performed markedly worse on darker-skinned female subjects than on lighter-skinned male subjects, with intersectional error rates as high as 34.7% for the worst-performing subgroup against as low as 0.8% for the best-performing one [1]. What we take from this is both the empirical case for facial-analysis auditing in the first place and the methodological template it uses, stratifying error rates by demographic subgroup and by demographic intersection rather than reporting a single aggregate accuracy figure. Where Gender Shades is a single, manual, point-in-time external audit of three commercial services, BiasAperture is designed to be the reusable software artefact that a Gender-Shades-style analysis was never built as: an auditor can point it at an arbitrary facial-analysis model and it repeats the same subgroup and intersectional comparison automatically, with significance testing Gender Shades’ original methodology did not itself formalise.

Dehdashtian et al. survey the broader computer-vision fairness literature, cataloguing where bias enters the pipeline, data collection, annotation, training, and deployment, and the families of fairness metrics and mitigation techniques the field has proposed at each stage [2]. This survey is what tells us the shape of the gap BiasAperture fills: it shows that a rich taxonomy of *what* to measure and *how* to mitigate already exists, but that comparatively little of the literature standardises *how* to run that measurement repeatably as a workflow. Rather than proposing another mitigation technique or another metric, our project sits deliberately on the diagnostic side of the taxonomy Dehdashtian et al. draw, building the reusable auditing pipeline their survey shows is largely absent, while explicitly declining to fold mitigation into the same platform.

Hardt et al. introduce the formal definitions of equalized odds and equal opportunity, two error-rate-based fairness criteria for supervised classifiers [9]. Equalized odds requires parity in both the true-positive rate and the false-positive rate across demographic groups,

while equal opportunity relaxes this to parity in the true-positive rate alone for the positive outcome class. What we learn from this paper is that these are two genuinely different fairness questions rather than one being a stricter version of an otherwise-interchangeable other, so neither is a general substitute for the other. Our project adopts both definitions directly as two of the Core Four metrics, EOD and EOP, computing them side by side rather than picking one, and pairs them with DPD and DIR to supply the outcome-rate and ratio-based perspectives that Hardt et al.’s error-rate framing does not itself cover.

Watkins et al. critique the U.S. four-fifths rule as it has been imported into algorithmic fairness practice, arguing that treating an 80% disparate-impact threshold borrowed from employment-discrimination law as a binary pass/fail test is an instance of epistemic trespassing that can grant a false sense of legal and ethical safety while conflating bare compliance with actual fairness [10]. This paper is the direct reason BiasAperture never reports a disparity as a ratio against an implicit cut-off. We differ from the naive-threshold practice the paper criticises by pairing every reported DIR value, and every other Core Four metric, with a chi-squared test of independence and a bootstrap confidence interval, so that a disparity is characterised by its statistical strength rather than measured against a borrowed and contested threshold.

Kurian et al. study how convolutional networks trained on medical images can learn to encode demographic information as a proxy through unrelated, seemingly neutral visual features, such as texture or lighting, even when no demographic label appears anywhere in the training data [11]. What we learn from this, applied to facial analysis, is that a detected subgroup disparity cannot be assumed to arise only from the demographic label an audit is stratifying on; it may instead be an artefact of a correlated visual feature the model has entangled with that label. Our project adopts this finding directly by requiring SHAP-based feature attribution on every flagged disparity rather than treating a Core Four result as self-explanatory, so the explainability layer can help distinguish a genuinely demographic effect from an unrelated confound before a finding is reported as evidence.

Mitchell et al. propose Model Cards, a standardised, structured summary format for a model’s intended use, performance characteristics, and evaluation results, aimed at making that information legible to both technical and non-technical stakeholders [6]. We learn from this paper a documentation convention the field already recognises, rather than needing to invent a report format from scratch. Our project adopts the Model Cards structure directly for the model-performance side of the generated fairness report, so that a reader familiar with Model Cards from elsewhere does not have to learn an ad hoc format specific to BiasAperture.

Gebru et al. propose the complementary Datasheets for Datasets convention, documenting a dataset’s motivation, composition, collection process, and recommended uses so that downstream users can reason about its limitations [7]. This paper supplies the dataset-side counterpart to Model Cards that our own report structure needed. We adopt the Datasheets convention for the dataset-provenance section of the generated report, so that FairFace’s and UTKFace’s collection processes and known limitations, such as UTKFace’s model-estimated age labels, are documented in a recognised structure rather than an ad hoc one.

Karikkainen and Joo construct FairFace, a face-attribute dataset explicitly built for balanced representation across race, gender, and age, using a seven-category race taxonomy designed to reduce the classification skew present in earlier face datasets [8]. What we learn from this paper is that FairFace’s balanced construction makes it a more trustworthy basis for subgroup comparison than a dataset whose subgroup sample sizes are skewed by construction. Our project adopts FairFace as its principal validation benchmark for exactly this reason, and uses UTKFace only as a secondary benchmark, since UTKFace’s age labels are model-estimated rather than human-annotated, a source of label noise the platform’s sample-size and confidence-interval reporting is designed to make transparent rather than obscure.

Buscemi et al. decompose the EU AI Act’s high-risk-system obligations into a stable taxonomy of high-level requirements and sub-requirements, mapping each to a concrete, technology-agnostic technical verification activity [12]. This is the paper that shows us how to turn a legal article into a specific, checkable test rather than a general statement of intent. Our project follows the same operational spirit at the scale of a single Article 10 mapping: Section 4.5 ties each Core Four metric, each significance test, and each sample-size guard to the specific data-governance clause it evidences, rather than the generated report gesturing at compliance in general terms.

Table 2-1. Summary of the literature reviewed, highlighting key contributions and motivations for this work.

Authors	Paper Title (Year)	Key Contribution	Motivation / Adopted Concepts
Buolamwini & Gebru [1]	Gender Shades (2018)	Audits commercial gender classifiers and reveals intersectional accuracy disparities by skin tone and gender.	Established the empirical case and subgroup/intersectional audit methodology; motivated building this into reusable software rather than a one-off audit.
Dehdashtian et al. [2]	Fairness and Bias Mitigation in Computer Vision: A Survey (2024)	Surveys where bias enters the CV pipeline and catalogues fairness-metric and mitigation families.	Showed the auditing-workflow literature is immature relative to the metrics literature; motivated the reusable, diagnostic-only pipeline.
Hardt et al. [9]	Equality of Opportunity in Supervised Learning (2016)	Formally defines equalized odds and equal opportunity as distinct fairness criteria.	Adopted directly as the EOD and EOP Core Four metrics, computed together rather than one substituting for the other.
Watkins et al. [10]	The Four-Fifths Rule is Not Disparate Impact (2022)	Critiques the four-fifths threshold as epistemic trespassing when used as a binary pass/fail test.	Motivated pairing every DIR value, and every Core Four metric, with a chi-squared test and bootstrap confidence interval instead of a bare threshold.

Continued on next page

Authors	Paper Title (Year)	Key Contribution	Motivation / Adopted Concepts
Kurian et al. [11]	Where, Why, and How is Bias Learned in Medical Image Analysis Models? (2024)	Shows CNNs can encode demographic information as a proxy through unrelated visual features.	Motivated the SHAP-based explainability layer, run on every flagged disparity to separate genuine demographic effects from confounds.
Mitchell et al. [6]	Model Cards for Model Reporting (2019)	Proposes a standardised, stakeholder-legible summary format for model performance and evaluation.	Adopted as the structural basis for the model-performance section of the generated fairness report.
Gebru et al. [7]	Datasheets for Datasets (2018)	Proposes a documentation convention for dataset motivation, composition, and limitations.	Adopted as the structural basis for the dataset-provenance section of the generated report.
Karkkainen & Joo [8]	FairFace (2021)	Constructs a face-attribute dataset with balanced race, gender, and age representation.	Adopted as the platform's principal validation benchmark, used to cross-check UTKFace's noisier, model-estimated age labels.
Buscemi et al. [12]	Assessing High-Risk AI Systems Under the EU AI Act (2025)	Maps EU AI Act high-risk obligations to concrete, technology-agnostic technical verification activities.	Motivated the Article 10-to-metric traceability mapping (Section 4.5) that ties each Core Four metric to a specific data-governance clause.

With everything above considered, BiasAperture combines what none of these works do alone: a reusable software pipeline rather than a one-off manual audit; the Core Four metrics computed by cross-validating, independently maintained toolkits rather than a single implementation; every disparity paired with the significance testing Watkins et

al.'s critique shows is necessary rather than a bare threshold; SHAP-based diagnosis of the proxy-variable risk Kurian et al. identify; a report structured against the Model Cards and Datasheets conventions; and explicit, clause-level traceability to the EU AI Act and NIST AI RMF in the operational spirit Buscemi et al. establish. Chapters 3 and 4 specify exactly this combination as BiasAperture's design, closing the engineering gap identified in Chapter 1.

3. REQUIREMENT ANALYSIS

This chapter specifies what BiasAperture must do, its Functional Requirements (FRs), and how well it must do it, its Non-Functional Requirements (NFRs). Hardware and software requirements are given as *system requirements*: what platform and environment are needed to build and run the system. Chapter 4 explains *how* these requirements are implemented and validated.

3.1. Functional Requirements

Functional requirements are listed using IEEE 830-style “shall” statements, one capability per requirement.

1. **FR-001 (Data Ingestion):** The system shall load, validate, and standardise a benchmark dataset (FairFace or UTKFace) or a compatible custom dataset, checking image integrity and aligning demographic annotation fields (race or ethnicity, perceived gender, age group) into a common internal schema.
2. **FR-002 (Dual-Mode Model Interface):** The system shall obtain predictions from a target facial-analysis model either by direct in-process inference against a supplied PyTorch or TensorFlow model object, or by batch ingestion of a precomputed predictions file in Comma-Separated Values (CSV) or JavaScript Object Notation (JSON) format.
3. **FR-003 (Fairness Metric Computation):** The system shall compute, per subgroup and per demographic intersection, the four Core Four disparity metrics, DPD, EOD, EOP, and DIR, using AIF360 and Fairlearn as independent, cross-validating computational backends.
4. **FR-004 (Statistical Significance Testing):** The system shall accompany every reported disparity with a chi-squared test of independence between the predicted-label distribution and the demographic subgroup, and with a bootstrap confidence interval on the subgroup accuracy difference.
5. **FR-005 (Explainability):** The system shall generate SHAP-based feature-attribution output identifying which input features are most associated with a detected disparity, to support diagnosis of proxy-variable entanglement.
6. **FR-006 (Report Generation):** The system shall assemble the computed metrics, statistical tests, and dataset and model provenance information into a standardised, exportable HTML fairness report, structurally informed by the Model Cards and Datasheets for Datasets conventions, with every metric row showing subgroup sample size, point estimate, 95% confidence interval, and either a p -value or an insufficient-sample flag.

7. **FR-007 (Regulatory Traceability):** The system shall map every computed metric, in the generated report, to its corresponding clause under Article 10 or Annex IV of the EU AI Act and to its corresponding function within the NIST AI RMF.
8. **FR-008 (Orchestration and Configuration):** The system shall expose a command-line configuration interface through which an auditor selects the benchmark dataset, the model or predictions source, and the metrics to compute, and shall re-run identically given the same configuration and inputs.
9. **FR-009 (Licensing Acknowledgement):** The system shall display and require explicit acknowledgement of the selected dataset’s licensing terms before an audit is executed.

3.2. Non-Functional Requirements

1. **NFR-001 (Statistical Rigour):** All significance testing shall use a threshold of $\alpha = 0.05$, and the system shall report exact p -values rather than only a pass/fail flag.
2. **NFR-002 (Uncertainty Quantification):** Every reported subgroup accuracy difference shall include a 95% confidence interval computed from a minimum of 1,000 bootstrap resamples.
3. **NFR-003 (Data-Integrity Guard):** Any subgroup with a sample size below $n = 30$ shall be automatically flagged as “insufficient sample, not reported” rather than assigned a computed metric value.
4. **NFR-004 (Performance):** A full-dataset evaluation over FairFace’s 97,698 released images (86,744 train + 10,954 validation) shall complete within 4 hours on a single mid-range Graphics Processing Unit (GPU); a stratified development subset of $n = 5,000$ images shall complete within 30 minutes on Central Processing Unit (CPU) alone.
5. **NFR-005 (Modularity):** The system shall maintain strict separation of concerns across its architectural modules, each exposing a narrow, well-defined interface, so that any module can be unit-tested and extended independently of the others.
6. **NFR-006 (Reproducibility):** Bootstrap resampling shall use fixed random seeds, and third-party dependency versions (AIF360, Fairlearn, PyTorch or TensorFlow) shall be pinned in a lock-file to prevent silent behavioural drift between runs.
7. **NFR-007 (Portability):** The system shall run cross-platform on any environment supporting the pinned Python dependency stack, without requiring proprietary or platform-specific components.
8. **NFR-008 (Explainability Performance):** SHAP attribution shall be computed only on flagged disparities rather than the full dataset by default, to keep explainability overhead a small fraction of total runtime.

3.3. System Requirements

This section answers what machine and software stack are required to build and run BiasAperture.

3.3.1. Hardware Requirements

- **Minimum (development, stratified subset):** 64-bit CPU, 4 or more cores, 8 GB Random-Access Memory (RAM), 5 GB free disk space.
- **Recommended (full-dataset audit, NFR-004 target):** A single mid-range GPU with 8 GB or more Video Random-Access Memory (VRAM), for example an NVIDIA T4-class device, either a local workstation GPU or a free-tier cloud notebook GPU such as Google Colab.
- No specialised or custom hardware is required; the platform performs inference and metric computation only, never model training from scratch.

3.3.2. Software Requirements

- **Operating system:** Linux, Windows, or macOS; the stack is pure Python and cross-platform.
- **Language and runtime:** Python 3.10 or later.
- **Model interface:** PyTorch or TensorFlow, whichever framework the audited model was built in.
- **Image preprocessing:** OpenCV, pandas, NumPy.
- **Fairness computation:** AIF360, Fairlearn, scikit-learn, SciPy.
- **Explainability:** SHAP.
- **Report generation:** Jinja2 for HTML templating.

4. SYSTEM ARCHITECTURE AND METHODOLOGY

Chapter 3 specified *what* BiasAperture must do. This chapter specifies *how*: the architectural decomposition and design principles that satisfy those requirements, the mapping from computed metrics to specific regulatory clauses, the work breakdown and schedule under which the system is to be built, and the verification strategy that gives the resulting fairness findings statistical credibility.

4.1. Design Principles

The architecture follows Object-Oriented Software Engineering practice under Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, Dependency Inversion (SOLID) guidelines, applied concretely rather than generically, so that each principle maps onto a specific structural decision rather than remaining an abstract design ideal. The single-responsibility principle is realised directly by the five-module split in Section 4.2: each module owns exactly one concern, ingestion, model access, metric computation, explanation, or reporting, and no module reaches into another's internal state. The open-closed principle is realised by the Strategy pattern around the fairness-metrics back end (Section 4.4): the Core Four disparity metrics, and the AIF360 and Fairlearn backends that compute them, are implemented as interchangeable strategies behind a common `FairnessBackend` interface, so a future fifth disparity metric, or a replacement fairness library, is added by implementing a new strategy without modifying the computation engine's internals. The Liskov substitution and interface segregation principles are satisfied by the narrowness of these same interfaces: because `ModelInterface` and `FairnessBackend` each expose only the minimal operations their callers use, any concrete implementation, whether `DirectInferenceAdapter`, `PredictionsFileAdapter`, `AIF360Backend`, or `FairlearnBackend`, can be substituted for another without altering the correctness of the code that depends on the interface. The dependency-inversion principle is realised by having the engine and report generator depend only on the abstract `FairnessBackend` and `ModelInterface` interfaces, never on AIF360, Fairlearn, or a specific file format directly, so that a cut or substitution at either boundary (Section 4.8) changes an implementation, not a contract. Finally, the shared schema described in Section 4.7 is an application of interface-first, design-by-contract practice: the field set two independently developed streams must both honour is fixed before either stream begins substantive work, allowing the two streams to be built and unit-tested in parallel against a shared specification rather than against each other's evolving implementation.

4.2. Architectural Modules

This section presents the system architecture of BiasAperture and the specific modules that realise it. The platform is composed of a set of cooperating modules, each responsible for one stage of the audit pipeline, coordinated by a central orchestration layer. Fig. 4-1 depicts this architecture as a whole, showing how external datasets and target models enter the system, how they are processed through the pipeline, and how the resulting compliance report is produced.

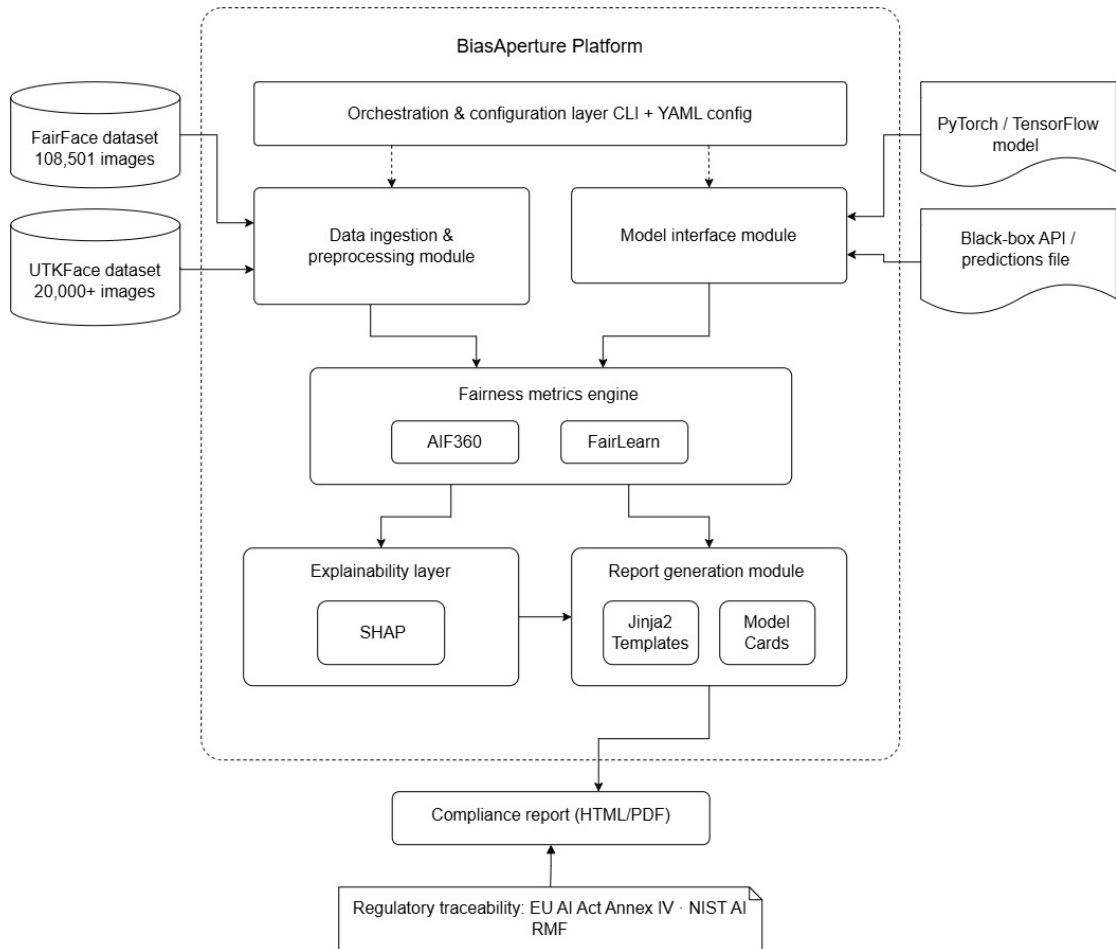


Figure 4-1. BiasAperture system architecture.

Table 4-1 formalises the architecture shown in Fig. 4-1, listing each module alongside its primary responsibility, the functional requirements it satisfies, and the key libraries it depends on. The remainder of this section discusses each module in turn, following the order in which data and control flow through the pipeline.

Table 4-1. Architectural modules and their responsibilities.

Module	Primary Responsibility	Key Libraries
Data Ingestion and Preprocessing	Load, validate, and standardise a benchmark or custom dataset (FR-001)	OpenCV, pandas, NumPy
Model Interface	Obtain predictions via direct inference or a predictions file (FR-002)	PyTorch, TensorFlow
Fairness Metrics Engine	Compute subgroup and intersectional metrics with significance testing (FR-003, FR-004)	AIF360, Fairlearn, scikit-learn, SciPy
Explainability Layer	Attribute a detected disparity to input features (FR-005)	SHAP
Report Generation	Assemble findings into a Model-Cards- and Datasheets-structured report (FR-006, FR-007)	Jinja2
Orchestration and Configuration	Coordinate the pipeline end to end behind a single entry point (FR-008, FR-009)	Python Command-Line Interface (CLI)

4.2.1. Data Ingestion and Preprocessing Module

This module loads a selected benchmark dataset, initially FairFace and UTKFace, or a compatible custom dataset, validates image integrity, standardises image resolution and colour space via OpenCV, and aligns demographic annotation fields (race or ethnicity, perceived gender, age group) into the locked internal schema (Section 4.7). It rejects corrupted or unreadable files and enforces that every image carries a complete, correctly typed set of demographic labels before any downstream computation proceeds.

4.2.2. Model Interface Module

This module is a thin, framework-agnostic adapter that obtains predictions from a target model without requiring internal weight access. Two integration modes are supported: direct in-process inference against a supplied PyTorch or TensorFlow model exposing a standard predict interface, and batch ingestion of a precomputed predictions file in CSV or JSON format for models that cannot be executed locally. The module performs no training, fine-tuning, or weight modification of any kind.

4.2.3. Fairness Metrics Computation Engine

The analytical core of the platform. It computes per-subgroup and intersectional performance metrics (accuracy, precision, recall, F1 Score (F1), false-positive and false-negative rates) and the four Core Four disparity metrics, DPD, EOD, EOP, and DIR, using AIF360 and Fairlearn as independent, cross-validating computational backends: because both toolkits already implement, test, and document these metrics, the platform does not need to re-derive fairness statistics from first principles, and any discrepancy between the two backends' output on the same input surfaces an integration defect rather than a genuine result. The engine additionally performs the chi-squared significance testing and bootstrap resampling required by FR-004.

4.2.4. Explainability Layer

Motivated by the proxy-variable finding discussed in Chapter 2, this layer applies SHAP to input features associated with a disparity the metrics engine has flagged, producing feature-attribution output that helps distinguish a genuinely demographic effect from an artefact of an unrelated correlated variable such as lighting or background. Consistent with NFR-008, attribution is computed only on flagged disparities rather than across the full dataset by default.

4.2.5. Report Generation Module

This module assembles the computed metrics, statistical tests, and dataset and model provenance into a standardised, exportable HTML report, structurally informed by the

Model Cards and Datasheets for Datasets conventions discussed in Chapter 2. Every metric row shows subgroup sample size, point estimate, 95% confidence interval, and either a p -value or an insufficient-sample flag (FR-006), and every row is additionally traceable to its Article 10 or Annex IV clause and its NIST AI RMF function (FR-007, Section 4.5). Because the reporting layer consumes only the already-computed metrics dictionary, it is architecturally decoupled from the computation engine, which simplifies testing and prevents a reporting defect from affecting the correctness of the underlying analysis.

4.2.6. Orchestration and Configuration Layer

A lightweight layer coordinates the pipeline end to end, dataset selection, model connection, metric selection, and report generation, behind a single entry point, and exposes a configuration file and command-line interface through which an auditor specifies the audit parameters (FR-008). The dataset’s licensing terms must be displayed and explicitly acknowledged before an audit is executed (FR-009).

4.3. End-to-End Workflow of the System

Fig. 4-2 presents the end-to-end workflow followed by the system during a single audit run, tracing the sequence of steps and decision points from initial configuration through to the generation of the final compliance report.

The workflow begins with the auditor configuring the audit: specifying the benchmark or custom dataset, the target model, or a predictions file, and the metrics to compute. The system then branches on the model access mode. If a model object is supplied, predictions are obtained by direct in-process inference against the PyTorch or TensorFlow model; otherwise, predictions are obtained by batch ingestion of a precomputed CSV or JSON file. Both branches converge on a common path, in which the selected dataset, FairFace or UTKFace, is ingested and validated, and predictions are generated over the resulting test matrix.

With predictions and ground-truth labels in hand, the system computes the Core Four disparity metrics together with SHAP-based feature attribution. Before any metric is reported, each demographic subgroup is checked against the minimum sample-size guardrail: subgroups with fewer than thirty observations are flagged as an insufficient sample and excluded from the statistical checks that follow, while all other subgroups proceed directly. The remaining subgroups are then subjected to chi-squared significance testing and bootstrap confidence-interval estimation, after which the system assembles and generates the final compliance report, concluding the workflow.

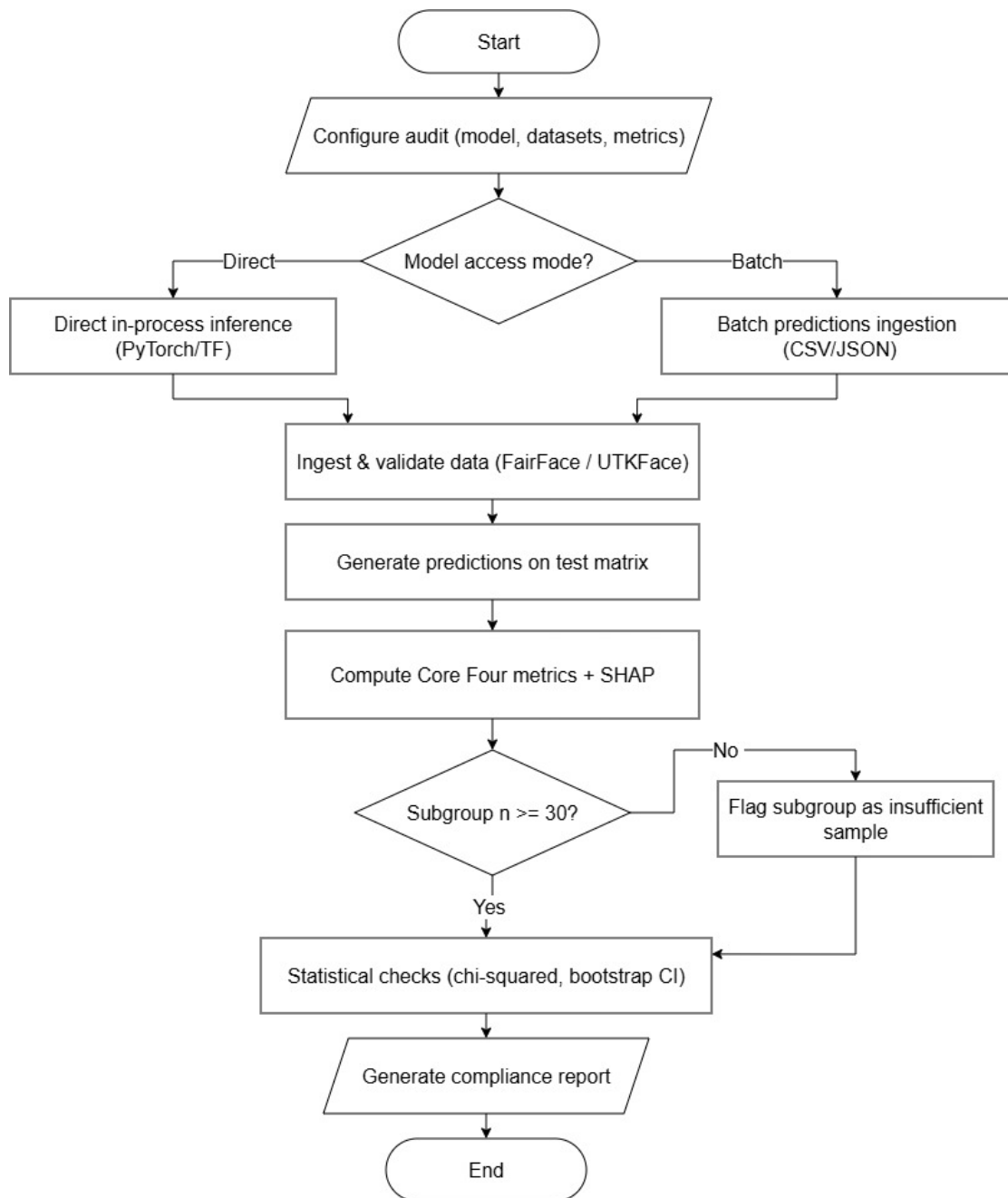


Figure 4-2. BiasAperture end-to-end audit workflow.

4.4. Design Patterns

Each pattern in Table 4-2 is tied to a specific project mechanic, either a descoping decision (Section 4.8) or the shared-schema contract (Section 4.7), rather than applied for its own sake.

Table 4-2. Design patterns mapped to architectural modules.

Pattern	Module	Structure	Rationale
Adapter	Model Interface	ModelInterface → DirectInferenceAdapter / PredictionsFileAdapter	Cutting direct inference (Section 4.8, item 4) means simply not instantiating one adapter; downstream code is unchanged
Strategy	Fairness Metrics Engine	FairnessBackend → AIF360Backend / FairlearnBackend	Cutting AIF360 (Section 4.8, item 5) is a backend swap, not a rewrite; the four metrics are interchangeable strategies
Builder	Data Ingestion	TestMatrixBuilder	Matches the multi-step curation (acquisition, integrity check, schema alignment) the module performs
Factory	Report Generation	ReportFactory → HTMLReportBuilder	Keeps the HTML report path untouched regardless of which export formats are in scope
Template Method	Report Generation	AuditReport base class	Fixes the Model Cards / Datasheets section skeleton while allowing subclasses to fill format-specific hooks
Facade	Orchestration	AuditOrchestrator	Hides the ingestion → interface → engine → report sequence behind one call

4.5. Regulatory Traceability Mapping

FR-007 requires that every computed metric be mapped to a specific regulatory basis. Table 4-3 shows how the EU AI Act’s Article 10 data-governance components map to the platform’s own outputs, following the operational-verification approach of Buscemi et al. discussed in Chapter 2 [3, 12].

The NIST AI RMF’s four functions map onto the project less as a report artefact than as a description of what the platform, and the process that built it, together accomplish. *Govern* is satisfied by the scope-boundary statement (Section 4.8) and TA-reviewed feasibility study that establish accountability for what the platform will and will not

Table 4-3. Article 10 components mapped to BiasAperture outputs.

Article 10 Component	Technical Requirement	BiasAperture Evidence
10(2) Governance	Documentation of design choices and data origin	Dataset provenance and licensing-acknowledgement log (FR-009)
10(3) Quality	Representativeness and error minimisation	Subgroup sample-size reporting and the $n \geq 30$ insufficiency guard (NFR-003)
10(4) Context	Geographical and behavioural adjustment	Per-dataset provenance section of the exported report
10(5) Mitigation	Bias detection and disparity evidence	The four Core Four disparity metrics with significance testing (FR-003, FR-004)

claim to do. *Map* is satisfied by the explicit demographic schema (Section 4.7) that identifies which subgroups the platform is designed to evaluate. *Measure* is the function the platform most directly operationalises: the fairness engine applies the Core Four metrics with significance testing to determine whether a subgroup disparity meets the defined statistical thresholds. *Manage* is out of the platform’s diagnostic scope by design, since BiasAperture reports disparities rather than prioritising or remediating them, and this exclusion is itself stated explicitly in every generated report.

4.6. Work Breakdown Structure

Table 4-4 decomposes the project into deliverable-oriented work packages following the 100% rule, every module in Section 4.2 is covered and nothing is added outside stated scope, with leaf packages sized for a two-person team and each leaf tied either to an acceptance criterion in Chapter 3 or to a module in Section 4.2. Ownership is tagged *Together* where a package requires shared judgement or iteration, or *Stream A / Stream B* where it can proceed solo; named assignment of which team member takes which solo stream remains open at time of writing and will be settled by demonstrated preference rather than fixed in advance.

4.7. Project Plan and Schedule

The work packages in Table 4-4 are restructured for concurrency rather than executed as a strict sequence, so that neither team member is idle while the other’s phase is blocking. Figure 4-3 shows the resulting eight-week schedule.

Table 4-4. Work breakdown structure.

ID	Work Package	Ownership
1.1	Project management: feasibility review, acceptance-criteria lock, schema lock, submission and defence prep	Together
1.2	Classifier selection and model interface: baseline confirmation, <code>ModelInterface</code> abstraction, predictions-file ingestion path	Together
1.3	Stream A, test-matrix construction: dataset acquisition and integrity check, inference run, schema curation, stratified development subset	Stream A
1.4	Stream B, report scaffolding: HTML and Jinja2 template, Model Cards / Datasheets structure, hand-written mock metrics dictionary	Stream B
1.5	Statistical detection engine: backend integration, four disparity metrics, chi-squared testing, bootstrap confidence interval, sample-size guard	Together
1.6	Integration: Stream A output into the detection engine, mock-to-real swap of Stream B's report against real findings, orchestration module	Together
1.7	Verification and validation: runtime targets, report-completeness rule, case-study run	Together
1.8	Documentation and delivery: scope-boundary statement, final report, submission package	Together

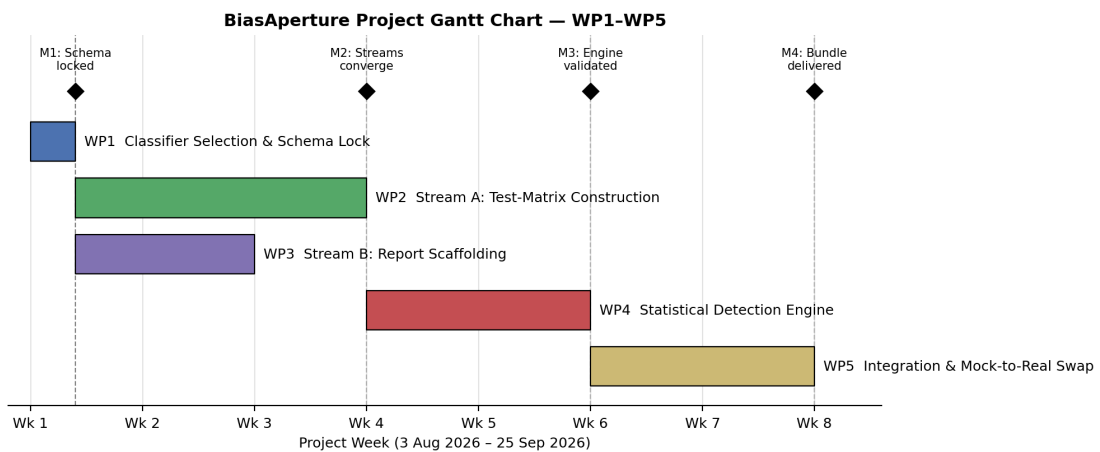


Figure 4-3. BiasAperture project schedule, work packages WP1–WP5.

WP1, *Classifier Selection and Schema Lock*, is a short, foundational, jointly worked phase that fixes the classifier baseline and the internal demographic schema every later module must honour: an `image_id`, race, gender, and age subgroup fields, predicted and true labels, and, for the detection engine’s output, metric name, point estimate, confidence bounds, p -value, and subgroup sample size. Its completion is Milestone M1, *schema locked*.

With the schema fixed, WP2 (Stream A, test-matrix construction) and WP3 (Stream B, report scaffolding) begin immediately and run in parallel rather than sequentially: Stream A curates the FairFace-based dataset and predictions file against the locked schema while Stream B builds the HTML report template and Model Cards / Datasheets structure against a hand-written mock metrics dictionary that validates against the same field set. Both streams converge at Milestone M2.

WP4, the *Statistical Detection Engine*, is the joint convergence phase: it consumes Stream A’s schema-aligned test matrix as input and produces the schema-shaped metrics dictionary Stream B’s report template already expects, running faster than it otherwise would because both interfaces were fixed before this phase began. Its completion, Milestone M3, is the point at which the engine is validated end to end.

WP5, *Integration and Mock-to-Real Swap*, replaces Stream B’s mock metrics dictionary with the detection engine’s real output, a short, mechanical step precisely because the template was built against an identical schema in week one, and completes with the final case-study audit and report bundle at Milestone M4.

4.8. Descoping Strategy

The scope defined in Chapter 3 is deliberately fuller than a two-person, eight-week timeline is guaranteed to accommodate without slippage. Rather than discover a descoping order under deadline pressure, Table 4-5 fixes it in advance.

Table 4-5. Descoping order if the project falls behind schedule.

#	Cut	Rationale
1	Web User Interface (UI), keep CLI only	Pure user-experience layer; no bearing on the fairness-engineering core
2	UTKFace, keep FairFace only	The architecture already anticipates this; UTKFace’s DEX-estimated label noise is an already-documented risk
3	Portable Document Format (PDF) export, keep HTML only	HTML alone satisfies the standardised, exportable report requirement and the Model Cards / Datasheets structure
4	Direct in-process inference, keep predictions-file ingestion only	The simpler mode; removes GPU and dependency-version risk entirely, and a public pretrained checkpoint and inference script make it sufficient for the case study
5	AIF360, keep Fairlearn only	Last resort; Fairlearn alone computes all four Core Four metrics, but this is the only cut that measurably weakens the cross-validated, methodologically defensible claim

The non-negotiable core that is never cut, regardless of timeline pressure, is: data ingestion, one model interface, the fairness engine, one report format, and the scope-boundary statement itself.

4.9. Verification and Validation Strategy

Statistical correctness is verified before any real-data run: the chi-squared and bootstrap-resampling routines are unit-tested against known reference values first, so that a defect in the significance-testing code is caught independently of whatever pattern happens to appear in a real audit’s results. Both streams’ outputs are additionally schema-validated against the locked field set from Section 4.7, directly defusing the risk that independent development on either side silently drifts and breaks the WP5 integration step. Version control follows a feature-branch-per-stream model off a `main` branch holding only the locked schema and module skeletons, with review at the WP4 convergence point before merge. Runtime and report-completeness are validated against the acceptance criteria already fixed in Chapter 3 (NFR-002, NFR-003, NFR-004),

and the case study itself, a FairFace-trained ResNet-34 baseline evaluated on all four Core Four metrics, exercises the full pipeline end to end as the platform's own validation instance.

5. CONCLUSION

5.1. Summary

This report has proposed BiasAperture, a diagnostic and evaluative software platform for auditing demographic accuracy disparities in third-party facial analysis models, and specified it in enough detail to be built against by a two-person team over an eight-week schedule. Chapter 2 showed that a mature fairness-metrics and mitigation-techniques literature exists without a corresponding reusable auditing artefact, at precisely the moment the EU AI Act’s Article 10 obligations make that gap a compliance question rather than only an academic one. Chapters 3 and 4 then specified how the platform closes it, against each of Chapter 1’s five specific objectives in turn: a modular architecture built around FR-001 and FR-002 satisfies the first; AIF360 and Fairlearn as independent, cross-validating backends for the four Core Four metrics, with chi-squared and bootstrap significance testing, satisfy the second; a report structured against the Model Cards and Datasheets conventions surveyed in Chapter 2 satisfies the third; validation against FairFace and UTKFace, with a FairFace-trained CNN baseline as the minimum case study, satisfies the fourth; and the Article 10 and NIST AI RMF mapping detailed in Section 4.5 satisfies the fifth.

Consistent with the scope stated in Chapter 1, BiasAperture remains strictly diagnostic throughout this design: it reports where a disparity exists and how statistically strong it is, and it does not mitigate bias, retrain a model, or generate synthetic demographic data. Section 4.8’s descoping order exists precisely so that this diagnostic core, ingestion, one model interface, the fairness engine, one report format, and the scope-boundary statement itself, is the one part of the proposal that is never at risk under timeline pressure, whatever else is cut first.

5.2. Future Work

The descoping order in Table 4-5 doubles as a map of the platform’s nearest extension points once the eight-week schedule in Section 4.7 is complete: a web UI over the existing CLI, PDF export alongside the HTML report, direct in-process inference restored alongside predictions-file ingestion, and UTKFace brought back in as a fully supported secondary benchmark once its label-noise limitation, discussed in Chapter 2, is more thoroughly characterised. Beyond the current cut-list, the clearest longer-term direction follows from the diagnostic and mitigation split Dehdashtian et al.’s survey draws [2]: BiasAperture is deliberately positioned on the diagnostic side of that split, and a natural next stage of the same research programme is a mitigation-facing companion that consumes BiasAperture’s findings as input rather than folding mitigation into the

diagnostic platform itself, keeping the separation of concerns this proposal establishes intact rather than eroding it.

A. PROJECT BUDGET

BiasAperture is a pure-software diagnostic platform with no bill of materials or physical hardware deliverable (Chapter 3, System Requirements); the budget below is scoped accordingly to cloud compute, subscriptions, and documentation costs rather than component procurement.

A.1. Cloud Computing and Development Services

Table A-1. Cloud computing and development services cost.

Service	Description	Cost
Open-source software	Python, PyTorch/TensorFlow, AIF360, Fairlearn, scikit-learn, SciPy, OpenCV, SHAP, Jinja2	Free
Cloud GPU compute (NFR-004 target)	Google Colab, free-tier T4-class GPU for full-dataset evaluation runs	Free
Cloud GPU compute (contingency)	Google Colab Pro, if free-tier quota is exhausted during the full 108,501-image FairFace evaluation (NFR-004)	\$10/month
Cloud storage	Google Drive, dataset and checkpoint storage during development	Free (existing quota)
Subtotal		\$0–\$10/month

A.2. Documentation and Miscellaneous

Table A-2. Documentation and miscellaneous cost.

Category	Item / Description	Estimated Cost
Printing and binding	Proposal, progress, and final report copies for submission and defence	\$10–\$15
Miscellaneous	Contingency for unplanned documentation or minor tooling needs	\$5–\$10
Total Estimated Cost		\$15–\$35 one-time, plus \$0–\$10/month cloud compute contingency

B. PROJECT TIMELINE

The project is executed over the eight-week schedule specified in Section 4.7, structured into work packages WP1–WP5 as detailed in Table 4-4. Figure B-1 reproduces the Gantt chart of Fig. 4-3 for appendix reference, showing task durations, the Stream A / Stream B concurrency, and milestones M1–M4.

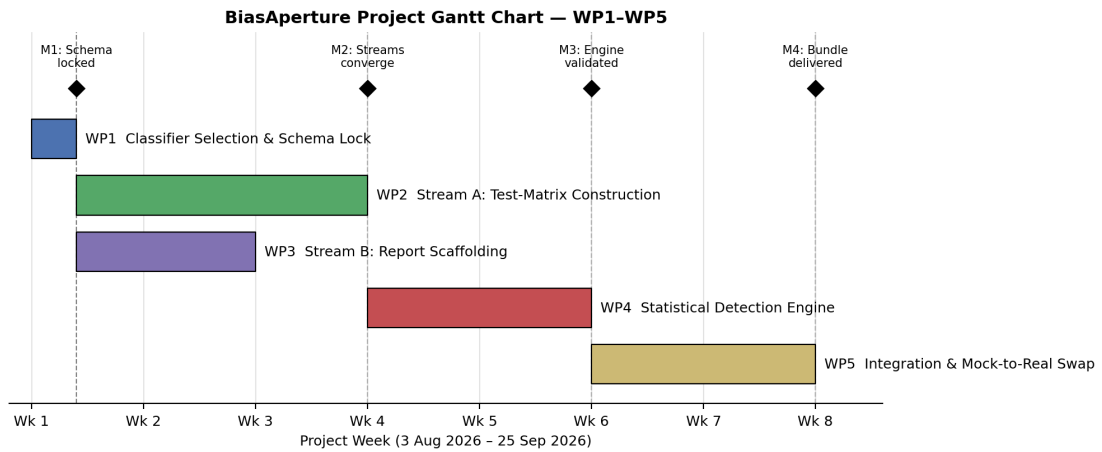


Figure B-1. BiasAperture project schedule, work packages WP1–WP5 (reproduced from Chapter 4).

C. LOCKED INTERNAL SCHEMA

Section 4.7 fixes the internal schema every module must honour before Stream A and Stream B begin parallel work. Tables C-1 and C-2 give the exact field-level specification of that schema.

Table C-1. Dataset / test-matrix schema.

Field	Type	Description
image_id	string	Unique identifier for the source image
race	categorical (7 levels)	FairFace race / ethnicity category
gender	categorical (binary)	Perceived gender label
age	categorical (9 bins)	FairFace age-group bin
true_label	categorical	Ground-truth demographic attribute value
predicted_label	categorical	Value returned by the Model Interface module
protected_attribute	categorical	Subgroup field currently selected for disparity analysis

Table C-2. Metrics-dictionary schema, the shape every Fairness Metrics Computation Engine output must satisfy.

Field	Type	Description
metric_name	string	One of DPD, EOD, EOP, DIR
subgroup_n	integer	Subgroup sample size
point_estimate	float	Computed metric value
ci_lower, ci_upper	float pair	95% bootstrap confidence interval bounds ($\geq 1,000$ resamples)
p_value	float	Result of the chi-squared significance test
insufficient_sample_flag	boolean	True when subgroup $n < 30$ (NFR-003)

D. RISK REGISTER

Table D-1 reproduces the risk register from the feasibility study underlying this proposal, covering the risks judged material enough to shape a design decision elsewhere in this report.

Table D-1. Project risk register: likelihood, impact, and mitigation strategy.

Risk	Likelihood	Impact	Mitigation
Demographic label noise, particularly UTKFace's model-estimated age labels	Medium	Medium	Report subgroup n and 95% confidence intervals alongside every metric; cross-validate against FairFace's manually annotated labels; document the limitation explicitly
Model interface incompatibility with non-standard prediction formats	Medium	Medium	Provide the documented CSV / JSON predictions-file ingestion path as a fallback to direct in-process inference
Misinterpretation of metrics by non-specialist stakeholders	Medium	High	Plain-language explanation for every metric, with explicit caveats on statistical significance and sample size, in the generated report
Scope creep toward mitigation or model retraining	Medium	High	Written scope document reviewed with the supervisor; report generation restricted to diagnostic, non-prescriptive output only
Licensing non-compliance for a custom user-supplied dataset	Low	High	Display and require explicit acknowledgement of licensing terms before an audit executes (FR-009)
Computational resource constraints delaying full-dataset evaluation	Low	Medium	Stratified development subset ($n = 5,000$) supported for development; expected hardware runtimes documented (NFR-004)
Dependency version drift across AIF360, Fairlearn, and the model framework	Medium	Low	Dependency versions pinned in a lock-file; periodic re-validation against upstream releases (NFR-006)
Dual-use risk: findings used to exploit rather than remediate a model's weaknesses	Low	Medium	Intended, permitted use stated explicitly in project documentation; platform restricted to diagnostic reporting only

REFERENCES

- [1] J. Buolamwini and T. Gebru, “Gender shades: Intersectional accuracy disparities in commercial gender classification,” in *Proceedings of the 1st Conference on Fairness, Accountability and Transparency (FAT*)*, ser. PMLR, vol. 81, 2018, pp. 77–91. [Online]. Available: <https://proceedings.mlr.press/v81/buolamwini18a.html>
- [2] S. Dehdashtian, R. Wang, and V. N. Boddeti, “Fairness and bias mitigation in computer vision: A survey,” *arXiv preprint arXiv:2408.02464*, 2024. [Online]. Available: <https://arxiv.org/abs/2408.02464>
- [3] European Parliament and Council of the European Union, “Regulation (eu) 2024/1689 laying down harmonised rules on artificial intelligence (artificial intelligence act),” 2024. [Online]. Available: <https://artificialintelligenceact.eu/>
- [4] —, “Regulation (eu) 2026/1744 (digital omnibus on artificial intelligence), deferring annex iii high-risk system obligations,” 2026. [Online]. Available: <https://artificialintelligenceact.eu/>
- [5] National Institute of Standards and Technology, “Artificial intelligence risk management framework (ai rmf 1.0),” National Institute of Standards and Technology, Tech. Rep. NIST AI 100-1, 2023. [Online]. Available: <https://www.nist.gov/itl/ai-risk-management-framework>
- [6] M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, and T. Gebru, “Model cards for model reporting,” in *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT* ’19)*. Atlanta, GA, USA: ACM, 2019, pp. 220–229.
- [7] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III, and K. Crawford, “Datasheets for datasets,” in *Proceedings of the 5th Workshop on Fairness, Accountability, and Transparency in Machine Learning*, ser. PMLR, vol. 80, Stockholm, Sweden, 2018, revised 2021. [Online]. Available: <https://arxiv.org/abs/1803.09010>
- [8] K. Karkkainen and J. Joo, “Fairface: Face attribute dataset for balanced race, gender, and age for bias measurement and mitigation,” in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2021, pp. 1548–1558. [Online]. Available: <https://arxiv.org/abs/1908.04913>
- [9] M. Hardt, E. Price, and N. Srebro, “Equality of opportunity in supervised learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 29, 2016,

pp. 3315–3323. [Online]. Available: <https://proceedings.neurips.cc/paper/2016/hash/9d2682367c3935defcb1f9e247a97c0d-Abstract.html>

- [10] E. A. Watkins, M. McKenna, and J. Chen, “The four-fifths rule is not disparate impact: A woeful tale of epistemic trespassing in algorithmic fairness,” *arXiv preprint arXiv:2202.09519*, 2022.
- [11] N. C. Kurian, P. Williams, S. Abdulrahman, J. Schrouff, R. Vandersluis, L. J. Palmer, and L. Oakden-Rayner, “Where, why, and how is bias learned in medical image analysis models? a study of bias encoding within convolutional networks using synthetic data,” *eBioMedicine*, 2024. [Online]. Available: <https://www.thelancet.com/journals/ebiom>
- [12] A. Buscemi, T. Deckenbrunnen, F. Kabir, K. Mishchenko, and N. Mowla, “Assessing high-risk AI systems under the EU AI act: From legal requirements to technical verification,” 2025. [Online]. Available: <https://arxiv.org/abs/2512.13907>